

IncluiDev: Desenvolvimento e Avaliação de uma Plataforma Baseada em Evidências para Apoio ao Ensino de Programação para Pessoas com Deficiência Visual

Lucas R. Pedro¹, Esteic Janaina Santos Batista¹

¹Faculdade de Computação – Universidade Federal de Mato Grosso do Sul (UFMS)
Campo Grande – MS – Brazil

lucas.pedro@ufms.br, esteic.batista@ufms.br

Abstract. *The inclusion of blind and visually impaired (BVI) individuals in Computer Science faces significant challenges due to the historical dependence on visual metaphors in programming languages and environments. Although studies, tools, and guidelines for accessible programming education exist, these resources remain scattered across the literature and the web. This study develops and evaluates a centralized repository, named IncluiDev, that organizes tools, methods, and scientific papers on the subject. The repository was built as an accessible web application and includes a Guided Assistant, a recommendation system based on user profiles (Students, Teachers, Researchers, and Developers). Accessibility compliance was assessed through automated audits (Google Lighthouse and WAVE) and manual testing with screen readers and keyboard navigation. As a preliminary heuristic step, the recommendation logic was also examined with AI-simulated personas (LLMs) guided by the items of the System Usability Scale (SUS) and the Technology Acceptance Model (TAM). Empirical validation with real participants is planned as future work.*

Resumo. *A inclusão de Pessoas com Deficiência Visual (PcDV) no campo da Ciência da Computação enfrenta desafios significativos devido à histórica dependência de metáforas visuais em linguagens e ambientes de programação. Apesar de existirem estudos, ferramentas e diretrizes voltados à acessibilidade no ensino de programação, esses recursos encontram-se dispersos na literatura e na web. Este estudo desenvolve e avalia um repositório centralizado, denominado IncluiDev, que organiza ferramentas, métodos e artigos científicos sobre o tema. O repositório foi construído como uma aplicação web acessível e incorpora um Assistente Guiado, um sistema de recomendação baseado em perfis de usuário (Estudantes, Professores, Pesquisadores e Desenvolvedores). A conformidade de acessibilidade foi avaliada por auditorias automatizadas (Google Lighthouse e WAVE) e por testes manuais com leitores de tela e navegação por teclado. Como etapa preliminar e heurística, a lógica de recomendação também foi examinada com personas simuladas por agentes de IA (LLMs), orientadas pelos itens das escalas System Usability Scale (SUS) e Technology Acceptance Model (TAM). A validação empírica com participantes reais está prevista como trabalho futuro.*

1. Introdução

A inclusão digital de Pessoas com Deficiência Visual (PcDV) nas áreas de Ciência da Computação e Tecnologia da Informação tem se tornado cada vez mais essencial em uma sociedade fortemente impulsionada pela tecnologia. O ensino de programação exige, tradicionalmente, a compreensão de metáforas visuais complexas, como diagramas de fluxo, blocos de código coloridos e a própria indentação textual [Resnick et al. 2009], o que cria barreiras de aprendizagem significativas para estudantes cegos ou com baixa visão.

A comunidade acadêmica e de desenvolvimento tem respondido a essas barreiras com a criação de linguagens baseadas em evidências, como a Quorum [Stefik et al. 2019], de ambientes tangíveis como o Code Jumper [Morrison et al. 2019] e da adaptação de leitores de tela em IDEs de mercado [Seo and Rogge 2023]. O desenvolvimento dessas soluções, porém, não garante por si só o acesso ao usuário final.

1.1. Problema de Pesquisa

Apesar do crescente número de estudos sobre o ensino de programação para PcDV, os recursos encontram-se dispersos, o que dificulta sua localização e seu uso prático pelos diferentes públicos. Professores em busca de metodologias de ensino, desenvolvedores em busca de diretrizes de acessibilidade e os próprios estudantes precisam reunir soluções fragmentadas em diversas fontes. Diante desse cenário, este trabalho busca responder à seguinte questão de pesquisa: como centralizar, classificar e recomendar de forma acessível metodologias, ferramentas e pesquisas sobre o ensino de programação para PcDV?

1.2. Objetivos

O objetivo geral deste trabalho é construir e avaliar um portal web dinâmico e centralizado que funcione como um repositório interativo de práticas, ferramentas e estudos voltados ao ensino de programação para PcDV.

Para alcançar esse propósito, foram definidos os seguintes objetivos específicos:

- Organizar os recursos identificados na literatura acadêmica e na indústria em uma taxonomia coerente;
- Desenvolver um repositório web em conformidade com as diretrizes de acessibilidade WCAG 2.1 nível AA [W3C 2018];
- Implementar um sistema de recomendação, o Assistente Guiado, baseado em jornadas de perfis (Estudante, Professor, Pesquisador e Desenvolvedor);
- Avaliar a usabilidade, a aceitação e a acessibilidade do sistema com métodos empíricos e escalas padronizadas [Brooke 1996, Davis 1989].

1.3. Organização do artigo

O restante deste artigo está organizado da seguinte forma. A Seção 2 apresenta o referencial teórico sobre o ensino acessível e os métodos de avaliação. A Seção 3 detalha os métodos aplicados na construção do repositório e nos procedimentos de validação. A Seção 4 reporta os resultados alcançados e a Seção 5 discute suas implicações. Por fim, a Seção 6 traz a conclusão e os trabalhos futuros.

2. Referencial Teórico

2.1. Ensino de Programação para Pessoas com Deficiência Visual

A compreensão de lógicas algorítmicas por indivíduos videntes apoia-se com frequência em recursos visuais, como diagramas de bloco, pseudocódigos formatados e interfaces de arrastar e soltar, a exemplo do Scratch [Resnick et al. 2009]. Para PcDV, a dependência quase exclusiva desses artefatos representa uma barreira significativa ao ensino e à prática de programação [Hadwen-Bennett et al. 2018].

Estudos na área apontam que a principal dificuldade reside na sobrecarga cognitiva imposta pela leitura sequencial dos leitores de tela [Baker et al. 2015]. Um programador vidente percebe a estrutura de um laço de repetição ou de uma condicional de forma global, por meio da indentação espacial, enquanto um usuário de leitor de tela precisa memorizar essa estrutura à medida que escuta o código linha a linha [Baker et al. 2019].

Para mitigar esses desafios, diferentes estratégias educacionais e tecnologias assistivas têm sido propostas. Abordagens baseadas em áudio e na sonificação de código, em que blocos e erros emitem sons característicos em um ambiente acústico, e ambientes tangíveis, como a programação física com hardwares modulares conectáveis [American Printing House for the Blind 2019], são exemplos de intervenções com eficácia demonstrada no ensino inclusivo [Stefik and Siebert 2013].

2.2. Recursos e Tecnologias para Inclusão

Duas estratégias distintas emergem na literatura: a adaptação de ferramentas já consolidadas e o desenvolvimento de novos ambientes e linguagens projetados para acessibilidade desde a origem. Ferramentas consolidadas no mercado, como o Visual Studio Code, passaram a oferecer modos de acessibilidade e atalhos otimizados que facilitam a navegação com leitores de tela [Seo and Rogge 2023, Microsoft 2024]. Já linguagens concebidas a partir de evidências de usabilidade, como a Quorum [Stefik et al. 2019], reduziram a dependência de caracteres especiais complexos e otimizaram a leitura fonética realizada por softwares de síntese de voz.

Apesar da crescente disponibilidade tecnológica, a aplicabilidade efetiva dessas ferramentas depende da curadoria da informação e do acesso a guias e diretrizes atualizados, como as Diretrizes de Acessibilidade para Conteúdo Web (WCAG) [W3C 2018]. A organização sistemática desses materiais em repositórios qualificados evita que educadores precisem criar adaptações pedagógicas do zero e promove o reuso e a padronização das intervenções de acessibilidade digital.

2.3. Avaliação de Sistemas Educacionais

A validação de portais e repositórios educacionais exige instrumentos confiáveis de avaliação. A *System Usability Scale* (SUS) [Brooke 1996] é um questionário de dez itens, padronizado internacionalmente, amplamente adotado para medir de forma rápida e quantitativa a usabilidade global de uma interface. O *Technology Acceptance Model* (TAM) [Davis 1989] complementa essa medida ao avaliar a utilidade percebida e a facilidade de uso percebida, fatores que determinam a intenção de uso e a adoção do sistema. No contexto da acessibilidade digital, esses modelos quantitativos, quando combinados a questões qualitativas abertas, fornecem evidências consistentes sobre as barreiras enfrentadas durante a interação e sobre o nível de satisfação do usuário.

3. Método

A presente pesquisa caracteriza-se como um estudo de natureza aplicada, conduzido sob uma abordagem de métodos mistos, quantitativa e qualitativa. O escopo do desenvolvimento foi definido a partir dos achados de uma revisão de literatura preliminar, que identificou e catalogou diretrizes metodológicas, ferramentas de software, aplicações lúdicas educacionais e estudos empíricos sobre a interseção entre o ensino de programação e a deficiência visual.

3.1. Desenvolvimento do Repositório

O repositório digital, denominado *IncluiDev*, foi projetado segundo a arquitetura de *Single Page Application* (SPA), com a biblioteca React [Meta Platforms, Inc. 2023], o empacotador Vite [You 2023] e o ambiente Node.js. A versão de produção da plataforma está disponível em <https://incluidev-portal.vercel.app/>.

A concepção arquitetural priorizou componentes funcionais acessíveis e a incorporação nativa de atributos WAI-ARIA [W3C 2023], como *aria-live*, para atualizações dinâmicas, e *aria-labelledby*, para estruturação semântica.

O processo contínuo de catalogação do estado da arte gerou uma taxonomia bi-dimensional, na qual cada recurso é indexado em função do Nível de Ensino (Básico, Técnico ou Superior) e do Público-Alvo primário.

As permissões, os papéis e as interações dos diferentes perfis com as ferramentas da aplicação estão representados no Diagrama de Casos de Uso (Figura 1).

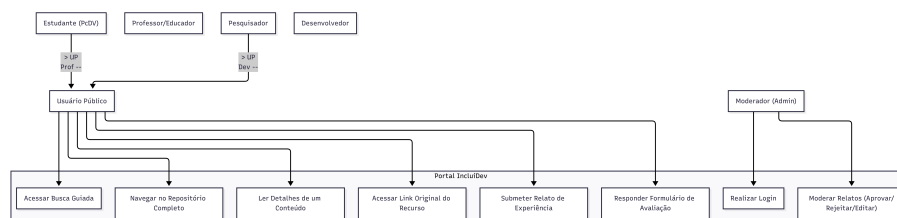


Figura 1. Diagrama de Casos de Uso do Portal IncluiDev

3.2. Construção do Assistente Guiado

Para superar as limitações dos mecanismos tradicionais de filtragem e de pesquisa em tabela, propôs-se o Assistente Guiado, um sistema de recomendação orientado pelo perfil do usuário. Em vez de uma listagem estática de filtros, o sistema conduz o usuário a uma autoidentificação a partir de quatro perfis levantados na literatura: Estudante, Professor, Pesquisador e Desenvolvedor.

Com base no perfil selecionado, a aplicação apresenta perguntas de múltipla escolha focadas no objetivo pedagógico ou técnico principal, como a exploração de ferramentas baseadas em áudio ou a obtenção de referências de revisões sistemáticas. O sistema então compara as respostas do usuário com as *tags* dos conteúdos cadastrados e apresenta recomendações relevantes sem sobrecarregar usuários de tecnologias assistivas.

O fluxo percorrido pelo usuário, da seleção de perfil até a exibição das recomendações, está ilustrado na Figura 2.

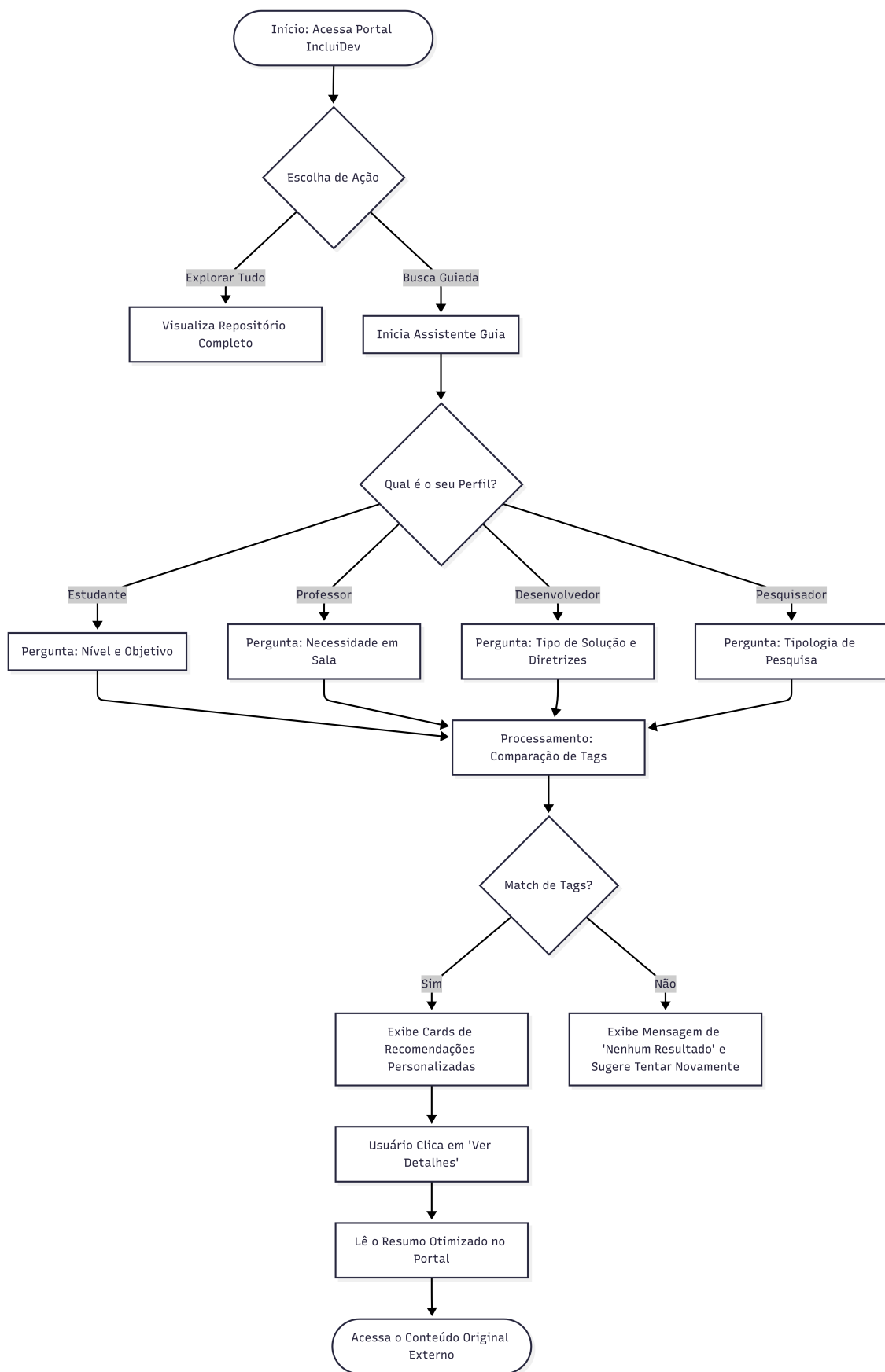


Figura 2. Fluxograma de Navegação do Assistente Guiado

3.3. Avaliação do Sistema

O protocolo de validação foi estruturado em fases. Como etapa preliminar, voltada à homologação da arquitetura lógica do sistema, empregou-se a simulação por agentes de IA (LLMs) como avaliadores heurísticos, técnica fundamentada na literatura recente sobre simulação de usuários em avaliações de interação humano-computador [Seshadri et al. 2026]. O mesmo estudo adverte que agentes de IA são *proxies* pouco confiáveis para usuários humanos, pois apresentam descalibração de julgamento e não vivenciam fadiga sensorial genuína. Por esse motivo, os resultados desta etapa são tratados como indicadores heurísticos preliminares, e não como medidas psicométricas validadas.

É necessário distinguir a forma de aplicação adotada. Em seu uso convencional, o SUS e o TAM são respondidos por usuários humanos, que atribuem a cada item uma pontuação em escala Likert, posteriormente convertida em escores numéricos. Neste trabalho, os itens dessas escalas não foram executados como questionário pontuado, mas serviram como roteiro de inspeção. Os agentes, instruídos a interpretar os quatro perfis de usuário (Estudante, Professor, Pesquisador e Desenvolvedor), interagiram com a plataforma e produziram respostas dissertativas orientadas por esses itens, abordando tanto a usabilidade da interface, dimensão central do SUS, quanto a utilidade e a facilidade de uso percebidas, dimensões do TAM. A simulação funciona, assim, como uma verificação heurística preparatória da coesão lógica e da acessibilidade técnica da aplicação, anterior à condução de testes empíricos com participantes reais, prevista como etapa seguinte do projeto.

4. Resultados

4.1. Caracterização do Repositório

A etapa de curadoria e extração consolidou um catálogo com 34 recursos, classificados segundo a taxonomia proposta. A categorização buscou abranger desde *frameworks* de alcance global, como as diretrizes de acessibilidade do W3C [W3C 2018] e a documentação MDN [Mozilla Developer Network 2024], até ambientes lúdicos e tangíveis voltados à iniciação no pensamento computacional sem o uso de monitor, como o Code Jumper [Morrison et al. 2019], e linguagens baseadas em evidências, como a Quorum [Stefik et al. 2019].

O portal reúne, em um único lugar, resumos detalhados de cada recurso com acesso direcionado às fontes científicas originais, de modo que professores e desenvolvedores localizem referências técnicas e legais sem recorrer a múltiplas buscas.

O código-fonte do sistema e os artefatos de validação descritos nas seções seguintes estão disponíveis no repositório público do projeto, em <https://github.com/lucasmantovany/incluidev-portal>; os registros da avaliação encontram-se na pasta `testes`.

A Figura 3 apresenta a tela inicial do portal, com o acesso rápido por perfis e níveis de ensino, e a Figura 4 mostra a tela do repositório, com a listagem de recursos e os filtros de conteúdo.

4.2. Funcionamento do Assistente Guiado

Durante os testes técnicos, a implementação do Assistente Guiado confirmou que as árvores de decisão e o cruzamento de *tags* retornam ao menos uma recomendação em

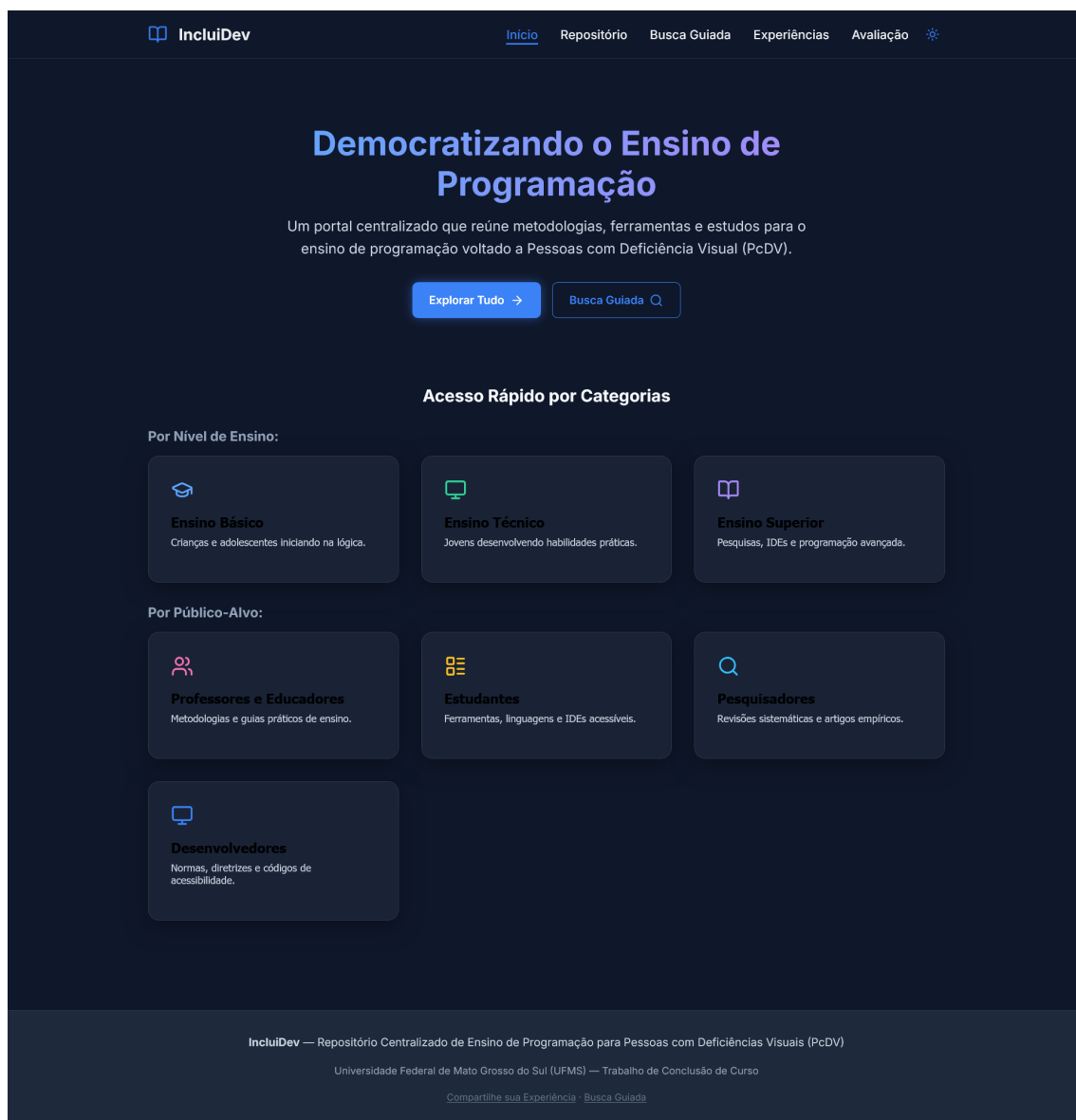


Figura 3. Tela Inicial do Portal InluidDev

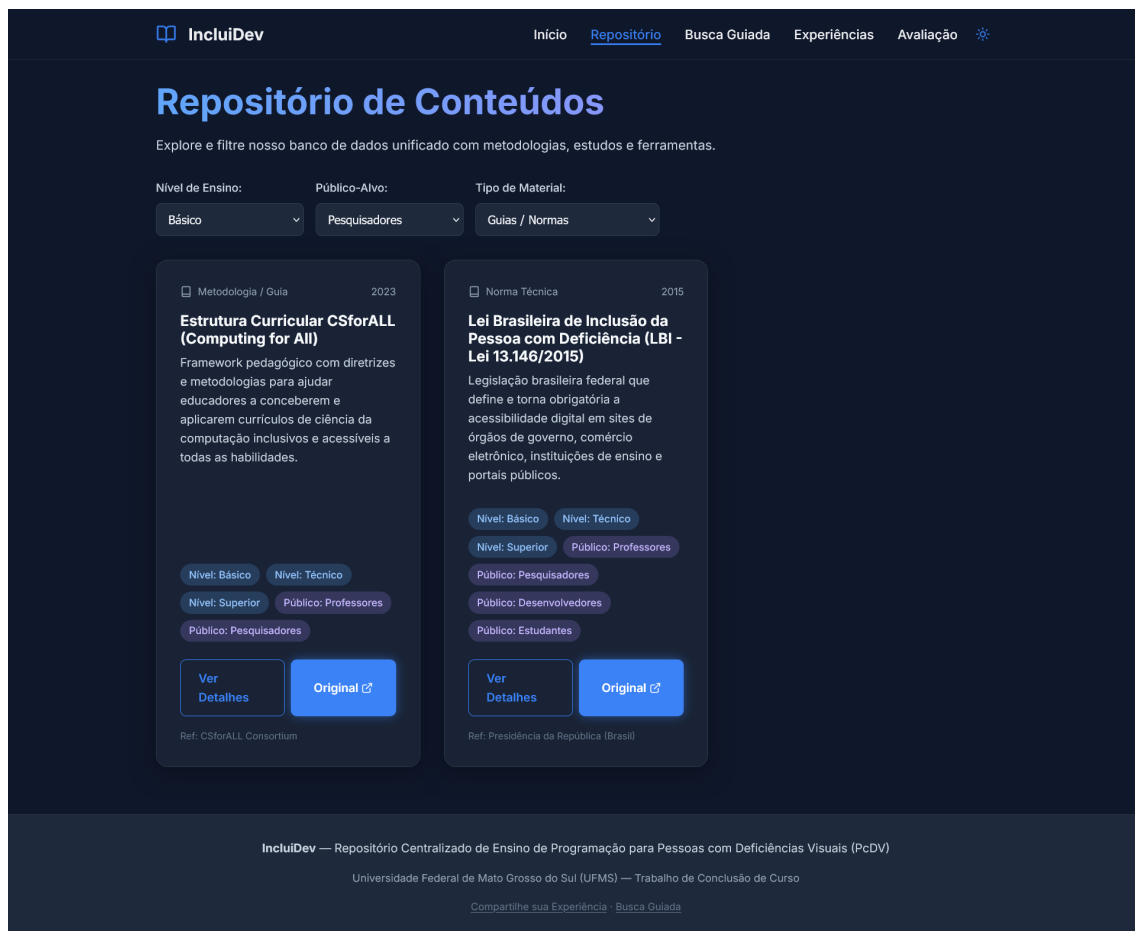


Figura 4. Tela do Repositório com Filtros de Conteúdo

todos os cenários cobertos pelo acervo atual. O fluxo respondeu de forma consistente às variações de contexto para cada um dos quatro perfis e manteve a compatibilidade com leitores de tela nos ambientes de teste. O arranjo visual limpo, sem *pop-ups* inesperados, favorece a navegação independente de usuários que dependem exclusivamente do teclado. A Figura 5 ilustra uma interação típica do Assistente, com as recomendações geradas a partir do perfil selecionado.



Figura 5. Recomendações geradas pelo Assistente Guiado a partir do perfil do usuário

4.3. Avaliação de Acessibilidade

Para verificar o cumprimento das diretrizes WCAG 2.1 (Nível AA), foram realizadas auditorias automatizadas e manuais. A avaliação automatizada com o Google Lighthouse apontou alta conformidade, e a ferramenta WAVE não identificou erros estruturais, registrando apenas alertas pontuais de contraste em elementos secundários. A Tabela 1 sintetiza os escores obtidos.

Tabela 1. Resultados das auditorias automatizadas de acessibilidade		
Ferramenta / Métrica	Desktop	Móvel
Google Lighthouse (escore de acessibilidade)	96/100	95/100
WAVE – erros estruturais	0	0
WAVE – alertas de contraste (elementos secundários)	Pontuais	Pontuais

Esses resultados, contudo, têm alcance limitado. Estudos de benchmarking de ferramentas de avaliação automatizada indicam que elas identificam apenas uma fração dos problemas de acessibilidade de uma interface, com cobertura estimada entre 14% e 38% dos critérios [Vigo et al. 2013]. Por isso, a auditoria manual, conduzida pelo próprio pesquisador com leitores de tela e navegação estrita por teclado, teve papel central na avaliação.

Na auditoria manual, verificou-se o funcionamento adequado da navegação por teclado, sem armadilhas de foco (*keyboard traps*). A aplicação emprega regiões dinâmicas (`aria-live="assertive"`) de modo a garantir que as mudanças de tela do Assistente Guiado sejam anunciadas de imediato por leitores de tela como o NVDA, o que evita a desorientação do usuário.

4.4. Validação Simulatória por Agentes de IA

Como etapa preliminar de validação estrutural e lógica da interface, a avaliação de usabilidade foi conduzida com agentes de IA simulando usuários (*LLM-Simulated Users*). Essa etapa busca assegurar a coesão do fluxo de navegação antes de submeter a plataforma a testes com usuários reais. Para isso, elaboraram-se *prompts* de sistema que simulam as quatro personas do projeto: Estudante, Professor, Pesquisador e Desenvolvedor. O registro completo do processo, com os *prompts* utilizados e as respostas geradas pelos quatro agentes, está disponível no repositório público do projeto, na pasta `testes/simulacao-agentes`, o que assegura a transparência metodológica da etapa.

A interação simulada das quatro personas com o Assistente Guiado resultou em respostas favoráveis aos itens do SUS e do TAM, sem que nenhuma persona relatasse obstáculos estruturais de navegação ou de compreensão do fluxo. Quanto à usabilidade aferida pelos itens do SUS, as justificativas apontaram baixa carga percebida na leitura sequencial. Quanto às dimensões do TAM, registrou-se alta utilidade percebida na recomendação direcionada por perfil e facilidade de uso na condução do fluxo. A persona de Estudante, por exemplo, registrou: “A interface fez perguntas objetivas. Não tive que ler centenas de links de pesquisa, o que aliviou minha fadiga com o NVDA”; a persona de Pesquisador comparou favoravelmente a busca guiada às consultas booleanas tradicionais.

A análise temática das quatro respostas revela um padrão recorrente entre os perfis de Estudante, Professor e Pesquisador: todos apontaram a redução do esforço de busca e de leitura sequencial como principal benefício, em contraste com a navegação por mecanismos de busca genéricos ou por bases bibliográficas extensas. A persona de Desenvolvedor enfatizou um eixo distinto, voltado à completude técnica e à ausência de ruído informacional na apresentação dos recursos. Esse contraste sugere que, embora o Assistente Guiado atenda de modo consistente à necessidade comum de reduzir a sobrecarga de navegação, versões futuras poderiam adotar critérios de relevância específicos por perfil, priorizando, por exemplo, profundidade técnica para Desenvolvedores e amplitude de fontes para Pesquisadores. Esses padrões derivam de uma simulação textual e devem ser tratados como hipóteses a confirmar com usuários reais, não como conclusões sobre o comportamento do público-alvo.

Convém reafirmar que esses resultados não substituem uma medição psicométrica.

Por se tratar de simulação textual, eventuais valores atribuídos pelos agentes às escalas SUS e TAM não têm a mesma validade de escores coletados com usuários humanos. Eles indicam apenas que a arquitetura lógica do sistema não apresenta inconsistências graves nos quatro fluxos avaliados, o que serve de base para a validação empírica planejada como trabalho futuro (Seção 6).

5. Discussão

Os achados desta primeira fase podem ser lidos à luz da literatura sobre a carga cognitiva imposta pelos leitores de tela. Estudos sobre a experiência de programadores cegos documentam o esforço considerável despendido para reconstruir mentalmente a estrutura do código durante a leitura sequencial [Baker et al. 2015, Baker et al. 2019]. O Assistente Guiado não atua sobre o código em si, mas transpõe esse princípio para a tarefa de busca: ao substituir a varredura de listas extensas por um diálogo curto orientado a perfil, reduz o volume de conteúdo que o usuário precisa percorrer e manter em memória. A baixa carga de leitura sequencial relatada nas simulações é coerente com essa expectativa, embora sua confirmação dependa de medição com usuários reais.

No plano do problema de pesquisa, os resultados sustentam a hipótese de que a centralização e a curadoria respondem à dispersão de recursos apontada na literatura [Hadwen-Bennett et al. 2018]. Ao reunir em um único ponto ferramentas, linguagens baseadas em evidências e diretrizes de acessibilidade [Stefik et al. 2019, W3C 2018], o *IncluiDev* reduz o custo de localização que recai sobre professores e desenvolvedores. A taxonomia bidimensional e a recomendação por perfil constituem, nesse sentido, menos uma inovação técnica e mais uma estratégia de organização de conhecimento já existente, alinhada à necessidade de que materiais de acessibilidade sejam reusáveis e padronizados em repositórios qualificados.

A principal cautela interpretativa recai sobre a natureza da avaliação. Os indícios de usabilidade e aceitação derivam de personas simuladas, e não de usuários humanos, o que restringe seu alcance a uma verificação de coerência lógica. Essa limitação é reconhecida pela própria literatura que fundamenta o método, pois agentes de IA tendem a julgamentos descalibrados e otimistas e não reproduzem a experiência sensorial do público-alvo [Seshadri et al. 2026]. Por esse motivo, os sinais favoráveis quanto às dimensões do SUS e do TAM devem ser entendidos como hipóteses a testar, não como evidência de aceitação. Soma-se a isso o fato de que a conformidade técnica medida pelas auditorias, ainda que combinada à inspeção manual, cobre apenas parte das barreiras reais de interação [Vigo et al. 2013]. Esses fatores delimitam a validade externa do estudo e reforçam que a etapa empírica com pessoas com deficiência visual é condição necessária para confirmar os benefícios aqui sinalizados.

6. Conclusão

A inclusão de Pessoas com Deficiência Visual no ensino de programação é um desafio complexo, agravado historicamente pela dispersão de ferramentas e metodologias acessíveis. Este estudo apresentou o *IncluiDev*, um repositório centralizado e dinâmico, projetado para reduzir essa fragmentação por meio de uma taxonomia estruturada e de uma navegação orientada a perfis de usuário.

As auditorias técnicas indicaram que a ferramenta atinge altos padrões de acessibilidade e cumpre as diretrizes da WCAG 2.1. A etapa preliminar de validação lógica, conduzida por simulação com agentes de IA, sugeriu que o Assistente Guiado oferece boa usabilidade e recomenda recursos pertinentes aos quatro perfis de usuário, com potencial para reduzir a carga cognitiva da navegação por leitores de tela.

As principais limitações do estudo decorrem da natureza preliminar da validação por agentes de IA, que não substitui a experiência de usuários reais, e da ausência de testes empíricos com participantes humanos, motivada pelo prazo da pesquisa. Soma-se a isso o fato, discutido na Seção 4.3, de que as ferramentas automatizadas de acessibilidade capturam apenas parte dos problemas reais de interação. Essas limitações apontam oportunidades de continuidade da pesquisa, entre as quais a realização de testes empíricos com grupos representativos de participantes, em especial pessoas com deficiência visual, tanto estudantes quanto docentes, e a comparação dos resultados simulados de SUS e TAM com avaliações conduzidas com usuários reais.

Ao reunir e organizar ferramentas e metodologias de programação inclusiva, o *IncluiDev* contribui para reduzir o tempo de busca de educadores e estudantes e para favorecer um ensino de programação mais inclusivo e estruturado. A expansão da base catalogada e o aprimoramento das heurísticas de recomendação configuram-se como outras oportunidades de evolução da ferramenta, capazes de consolidá-la como referência nesse ecossistema.

Referências

- American Printing House for the Blind (2019). Code jumper: A physical programming language for students who are blind or visually impaired. [https://www.aph.org/product/code-jumper/](https://wwwAPH.org/product/code-jumper/). Originalmente desenvolvido como Project Torino pela Microsoft Research.
- Baker, C. M., Bennett, C. L., and Ladner, R. E. (2019). Educational experiences of blind programmers. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE '19)*, pages 759–765. ACM.
- Baker, C. M., Milne, L. R., and Ladner, R. E. (2015). Structjumper: A tool to help blind programmers navigate and understand the structure of code. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI '15)*, pages 3043–3052. ACM.
- Brooke, J. (1996). Sus: A ‘quick and dirty’ usability scale. In Jordan, P. W., Thomas, B., Weerdmeester, B. A., and McClelland, I. L., editors, *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, London.
- Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3):319–340.
- Hadwen-Bennett, A., Sentance, S., and Morrison, C. (2018). Making programming accessible to learners with visual impairments: A literature review. *International Journal of Computer Science Education in Schools*, 2(2):3–13.
- Meta Platforms, Inc. (2023). React: A javascript library for building user interfaces. <https://react.dev/>. Acessado em: 2025.

- Microsoft (2024). Accessibility in Visual Studio Code. <https://code.visualstudio.com/docs/editor/accessibility>. Acessado em: 2025.
- Morrison, C., Cutrell, E., Dhareshwar, A., Villar, N., Thieme, A., Taylor, A., et al. (2019). Physical programming for blind and low vision children at scale. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pages 1–13.
- Mozilla Developer Network (2024). MDN web docs: Resources for developers, by developers. <https://developer.mozilla.org/>. Acessado em: 2025.
- Resnick, M., Maloney, J., Monroy-Hernández, A., Rusk, N., Eastmond, E., Brennan, K., Millner, A., Rosenbaum, E., Silver, J., Silverman, B., and Kafai, Y. (2009). Scratch: Programming for all. *Communications of the ACM*, 52(11):60–67.
- Seo, J. and Rogge, M. (2023). Coding non-visually in visual studio code: Collaboration towards accessible development environment for blind programmers. *Journal on Technology and Persons with Disabilities*, 11:120–136.
- Seshadri, P., Cahyawijaya, S., Odumakinde, A., Singh, S., and Goldfarb-Tarrant, S. (2026). Lost in simulation: Llm-simulated users are unreliable proxies for human users in agentic evaluations. *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*.
- Stefik, A., Ladner, R. E., Groce, A., and Hollings, S. (2019). The quorum programming language. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE '19)*. ACM.
- Stefik, A. and Siebert, S. (2013). An empirical investigation into programming language syntax. *ACM Transactions on Computing Education (TOCE)*, 13(4):1–40.
- Vigo, M., Brown, J., and Conway, V. (2013). Benchmarking web accessibility evaluation tools: Measuring the harm of sole reliance on automated tests. In *Proceedings of the 10th International Cross-Disciplinary Conference on Web Accessibility (W4A '13)*. ACM.
- W3C (2018). Web content accessibility guidelines (WCAG) 2.1. W3c recommendation, World Wide Web Consortium (W3C). Disponível em: <https://www.w3.org/TR/WCAG21/>.
- W3C (2023). Accessible rich internet applications (WAI-ARIA) 1.2. W3c recommendation, World Wide Web Consortium (W3C). Disponível em: <https://www.w3.org/TR/wai-aria-1.2/>.
- You, E. (2023). Vite: Next generation frontend tooling. <https://vitejs.dev/>. Acessado em: 2025.